

Catalog of 1D potentials in `codes/potentials/potentials_1d.py`

Generated with Claude Code, from a code-reading session on 2026-08-31

Scope and notation

This note catalogs every potential class in `codes/potentials/potentials_1d.py`, in file order, from `Scattering_First_Order_1d` up to and including `L2p1_norm`. `Scalar_GGD_KRegion` and everything after it (the generalized-Gaussian fitting potentials) are out of scope. Three infrastructure classes precede the catalog (`Potential`, `Potential_Prepere`, `Potential_Parallel`) — they are base/wrapper classes with no mathematical content of their own and are not covered here.

Filter banks. `get_1d_potentials` (`codes/potentials_builder.py`) builds every potential from one of three filter banks, all constructed by `return_Filters` (`codes/filters_bank.py:21`):

- **filters** — the *coarse* bank, $Q = 1$ wavelet per octave. **Always** includes one low-pass channel ϕ concatenated as its last channel, alongside J band-pass channels $\psi_1, \dots, \psi_J$: $J + 1$ channels total.
- **filters_Q** — the *fine* bank, Q wavelets per octave (whatever Q the run was configured with, e.g. $Q = 3$). Same structure: JQ band-pass channels plus the same one low-pass channel concatenated at the end: $JQ + 1$ channels total.
- **filters_Phi** — a *separate* stack of J different low-pass filters, one per scale $j = 1, \dots, J$ (increasingly smoothed). Not a duplicate of the low-pass channel bundled into **filters/filters_Q**.

Several call sites slice off the bundled low-pass channel explicitly with `[:, :-1]` when a potential should only see band-pass channels.

Wavelet coefficient. $Wx[\lambda](t) := (\psi_\lambda * x)(t)$, a complex-valued coefficient at scale/channel λ and time t (Morlet wavelets are complex). $\mathbb{E}[\cdot]$ denotes the empirical average over batch and time, $\frac{1}{BT} \sum_{b,t}$, unless stated otherwise. $\sigma(\cdot)$ denotes the logistic sigmoid; $\text{abs_eps}(z) = \sqrt{\text{Re}(z)^2 + \text{Im}(z)^2 + \varepsilon}$ (`utils_potentials.py:135`) is used with $\varepsilon = 0$ everywhere in this file, i.e. it is exactly $|z|$ in every call site checked.

The micro-normalization mechanism (`fit_micro/self.norm`). Several classes below define `fit_micro(x)`: called *once*, on real data, before fitting or generation (never on synthetic samples). It sets `self.norm`, a per-scale rescaling factor that `forward()` divides the *raw first-layer coefficient* by, before any further nonlinearity. The motivation (from the scattering-spectra literature): this codebase’s data has a power-law spectrum, so $\mathbb{E}[|Wx_s|^2]$ varies hugely across scales s ; without correcting for that, whichever scale carries the most raw power dominates the fitted θ purely because it is louder, and fitting becomes ill-conditioned across frequencies. The reference normalization,

generalizing Eq. 22 of the scattering-spectra paper (Morel et al.) — divide the wavelet coefficient by its own standard deviation once, and every statistic built from it inherits the correctly-cancelling power of σ :

$$Wx[\lambda] \mapsto Wx[\lambda]/\sigma[\lambda], \quad \sigma[\lambda] := (\mathbb{E}[|Wx[\lambda]|^2])^{1/2}. \quad (1)$$

Each subsection below states explicitly what `self.norm` actually is, and whether it matches (1) (verified, or fixed on 2026-08-31 during this session), or departs from it (flagged, unresolved).

1 Scattering_First_Order_1d

Code: `potentials_1d.py:81--106`. **Term:** `'Scattering_First_Order'`.

Formula.

$$S_1[\lambda] = \mathbb{E}[|Wx[\lambda](t)|], \quad \lambda = 1, \dots, JQ. \quad (2)$$

The plain first absolute moment (modulus) of the wavelet coefficient, per fine scale.

Filter bank. `potentials_builder.py:190: Scattering_First_Order_1d(filters_Q[:, :-1])` — the **fine** bank, **band-pass channels only** (the bundled low-pass is sliced off). Runs at whatever Q the experiment was configured with (e.g. $Q = 3$), *not* $Q = 1$. [\[verified from code\]](#)

Normalization. [\[none implemented\]](#). No `fit_micro` override — inherits the base `Potential.fit_micro`, a no-op. `self.norm` is never set, and `forward()` never references it. This statistic is exactly as (1) says it should *not* be left: fully exposed to whatever power each scale λ happens to carry.

2 Scattering_Second_Order_1d

Code: `potentials_1d.py:114--137`. **Terms:** `'Scattering_Second_Order'`, `'Scattering_Second_Order_Q1'`.

Formula.

$$S_2[\lambda] = \mathbb{E}[Wx[\lambda](t) \overline{Wx[\lambda](t)}] = \mathbb{E}[|Wx[\lambda](t)|^2]. \quad (3)$$

The plain second moment (power spectrum) of the wavelet coefficient, per scale.

Filter bank. Instantiated *twice* in `potentials_builder.py`:

- Line 193: `Scattering_Second_Order_1d(filters_Q)` — **fine** bank, term `'Scattering_Second_Order'`. *Includes* the bundled low-pass channel (no slicing here, unlike the first-order sibling above).
- Line 211: `Scattering_Second_Order_1d(filters)` — **coarse** ($Q = 1$) bank, term `'Scattering_Second_Order_Q1'`. Also includes the low-pass channel.

[\[verified from code\]](#)

Normalization. `[none implemented]`. Same as `Scattering_First_Order_1d`: no `fit_micro`, no `self.norm` division.

3 Scattering_Second_Order_Bulk_1d

Code: `potentials_1d.py:139--213`. **Term:** `'Scattering_Second_Order_bulk'` (comment in code: “experiment for lagrangian turbulence”; only on the distribution bulk).

Formula. A *trimmed* second moment: the same $|Wx[\lambda](t)|^2$ as above, but weighted by a smooth window that suppresses the tail of the distribution of $|\text{Re}(Wx[\lambda](t))|$ (note: the window is built from the *real part*’s magnitude, not the complex modulus, while the quantity being averaged *is* the full complex modulus squared). Calibrated once, from data, in `fit(x)`:

$$c[\lambda] = \text{the } \text{bulk_quantile} \text{ (default 0.90) quantile of } |\text{Re}(Wx[\lambda](t))| \text{ over all } (b, t), \quad (4)$$

$$s[\lambda] = \frac{1}{4}(Q_{0.95}(|\text{Re}(Wx[\lambda])|) - Q_{0.85}(|\text{Re}(Wx[\lambda])|)), \text{ clamped to } \geq 10^{-6}. \quad (5)$$

Then, per-sample:

$$w_{\text{bulk}}[\lambda](t) = 1 - \sigma\left(\frac{|\text{Re}(Wx[\lambda](t))| - c[\lambda]}{s[\lambda]}\right), \quad S_2^{\text{bulk}}[\lambda] = \mathbb{E}_t[w_{\text{bulk}}[\lambda](t) \cdot |Wx[\lambda](t)|^2]. \quad (6)$$

A robust/trimmed power spectrum: samples whose real part exceeds the bulk quantile threshold are smoothly down-weighted rather than hard-clipped.

Filter bank. `potentials_builder.py:197: Scattering_Second_Order_Bulk_1d(filters_Q)` — **fine** bank, full (band-pass + low-pass). [\[verified from code\]](#)

Normalization. Not a `fit_micro/self.norm`-style rescaling of the output — instead uses its own `fit(x)` method to calibrate the bulk/tail cutoff $(c[\lambda], s[\lambda])$ directly from data. The output itself is never divided by anything. This addresses a different problem (robustness to outliers/tails) than (1) (comparability across scales’ own power) — the two are not interchangeable, and this class still has none of the latter.

4 Scattering_Third_Order_Real_1d

Code: `potentials_1d.py:215--269`. **Term:** `'Scattering_Third_Order_Real'`. (`'Scattering_Third_Order_Imag'` is wired to a `pass` in `potentials_builder.py:203` — the corresponding `Scattering_Third_Order_Imag_1d` class is commented out and does not exist as a usable term.)

Formula. A genuine *cross-scale* correlation — the classic scattering third-order coefficient, correlating the coefficient *at the second-transform’s own scale* μ against a second wavelet transform, built at that same scale μ , of the modulus envelope from a finer (or equal) scale $\lambda \leq \mu$:

$$S_3[\lambda, \mu] = \mathbb{E}_t\left[\text{Re}\left(\overline{Wx[\mu](t)} \cdot (\psi_\mu * |Wx[\lambda](\cdot)|)(t)\right)\right], \quad \mu \geq \lambda. \quad (7)$$

[note corrected 2026-08-31 – code unchanged] Traced by hand from `forward()` (`potentials_1d.py:222--235`) and confirmed numerically against dummy tensors, since the broadcasting there is easy to misread: the coefficient that gets conjugated is $\overline{Wx[\mu]}$ (same scale as the second-layer filter). This is the standard scattering-transform convention (a fine-scale envelope, re-filtered at a coarser scale μ , correlated against the coefficient).

Filter bank and a real caveat at $Q > 1$. `potentials_builder.py:200: Scattering_Third_Order_Real_1d(J, filters_Q)` — fine bank, full (no slicing), with `self.J = J` (`args.J`, the base scale count, *not* $J*Q$) and `self.filters = filters_Q` ($JQ + 1$ channels). [verified from code] The index pairs (λ, μ) are generated by `indices_third_order(self.J, 1)`, which builds them assuming `num_filters = self.J+1` — *i.e. it only ever emits indices in $[0, J]$* , regardless of how many channels the bank passed to `forward()` actually has. [note corrected 2026-08-31 – code unchanged] I confirmed this numerically (`indices_third_order(J,1)` with $J=8, Q=3$ tops out at index 8, never 9–24) rather than trusting the shapes by eye.

- At $Q = 1$ (`filters_Q` then equals the coarse bank, $J + 1$ channels) this is exactly right: every channel of the bank passed in is reachable, and μ correctly ranges up to and including the low-pass channel. This is the regime this term is actually used in.
- At $Q > 1$ (the fine bank actually has $JQ + 1$ channels) this is **broken**: channels $J+1, \dots, JQ$ of `filters_Q` are silently never referenced by this potential at all. The term still runs without error and returns a plausible-looking tensor of the shape `self.num_coefficients = J((1+J)/2+1)` predicts — it just quietly ignores most of the fine bank’s scales rather than computing the cross-scale statistic over all of them. **Do not use ‘Scattering_Third_Order_Real’ with $Q > 1$ filters as currently implemented** — fixing it would mean calling `indices_third_order` with an argument reflecting the actual $JQ + 1$ channel count, not `self.J`.

Normalization. [none implemented]. No `fit_micro` defined.

5 Scattering_Fourth_Order_Real_1d

Code: `potentials_1d.py:273--398` (post-fix). **Terms:** ‘Scattering_Fourth_Order_Real’, ‘Scattering_Fourth_Order_Real_Q1’.

Formula. Correlates the second-layer (coarse-scale) wavelet transform of the *modulus* envelope $|Wx[s]|$ at two fine scales $s_1 \leq s_2$, at a shared coarse scale j :

$$Wx^{\text{norm}}[s](t) := Wx[s](t)/\sigma[s], \quad E[s](t) := |Wx^{\text{norm}}[s](t)|, \quad (8)$$

$$W_j E[s](t) := (\phi_j * E[s](\cdot))(t) \quad (\text{second, coarse-bank transform}), \quad (9)$$

$$S_4^{\text{Re}}[s_1, s_2, j] = \mathbb{E}_t \left[\text{Re}(W_j E[s_1](t) \cdot \overline{W_j E[s_2](t)}) \right]. \quad (10)$$

j ranges over *all* coarse channels including the low-pass one (`include_lowpass` defaults to `True` for this class — not passed explicitly at any call site). Only $s_1 \leq s_2$ is stored: $\text{Re}(z)$ is symmetric under swapping $s_1 \leftrightarrow s_2$, so the other half is redundant.

Filter bank. Two wirings, both in `potentials_builder.py`:

- Line 207: `Scattering_Fourth_Order_Real_1d(J, Q, filters, filters_Q[:, :-1])` — term `'Scattering_Fourth_Order_Real'`: second layer on the **coarse** bank, first layer on the **fine** bank (band-pass only), at whatever Q was configured.
- Line 216: `Scattering_Fourth_Order_Real_1d(J, 1, filters, filters[:, :-1])` — term `'Scattering_Fourth_Order_Real_Q1'`: both layers on the **coarse** ($Q = 1$) bank explicitly.

[verified from code]

Normalization. `self.norm[s] = (E[|Wx[s]|2])0.5 =  $\sigma[s]$` [fixed 2026-08-31] — matches (1) exactly. Previously `self.norm[s] = (E[|Wx[s]|2])-1/4`; the old line is kept commented out in the source. Derivation of why 0.5 is the exponent that cancels the scale-dependence of S_4^{Re} (independent of whether the envelope is $|Wx|$ or $|Wx|^2$, see Section 7): tracking how the raw (unnormalized) statistic scales with $E_{s_1} := \mathbb{E}[|Wx[s_1]|^2]$ and E_{s_2} gives a factor $E_{s_1}^{1/2-p} E_{s_2}^{1/2-p}$ for exponent p on $\sigma[s]$ in the denominator; this vanishes (scale-invariance) only at $p = 1/2$.

6 Scattering_Fourth_Order_Imag_1d

Code: `potentials_1d.py:400--513`. **Terms:** `'Scattering_Fourth_Order_Imag'`, `'Scattering_Fourth_Order_Imag_Q1'`.

Formula. Identical construction to Section 5, keeping the imaginary part instead:

$$S_4^{\text{Im}}[s_1, s_2, j] = \mathbb{E}_t \left[\text{Im}(W_j E[s_1](t) \cdot \overline{W_j E[s_2](t)}) \right], \quad s_1 < s_2 \text{ strictly.} \quad (11)$$

$\text{Im}(z)$ is *antisymmetric* under $s_1 \leftrightarrow s_2$ (swapping gives $\text{Im}(\bar{z}) = -\text{Im}(z)$), so the diagonal $s_1 = s_2$ is identically zero and is always excluded (`offset=1`, not optional here — unlike the Real class, this class has no `include_diag` constructor argument at all). **Also:** `include_lowpass=False` is hardcoded in this class's index construction, so j ranges only over the J band-pass coarse channels, *excluding* the low-pass one — a second, independent difference from the Real class (Section 5) beyond the diagonal exclusion.

Filter bank. Two wirings:

- Line 219: `Scattering_Fourth_Order_Imag_1d(J, Q, filters, filters_Q[:, :-1])` — term `'Scattering_Fourth_Order_Imag'`: fine first layer, coarse second layer.
- Line 226: `Scattering_Fourth_Order_Imag_1d(J, 1, filters, filters[:, :-1])` — term `'Scattering_Fourth_Order_Imag_Q1'`: both layers coarse.

[verified from code]

Normalization. `self.norm[s] =  $\sigma[s]$` (exponent 0.5, line 508). [verified from code] — already matched (1) before this session; not modified.

7 Scattering_Fourth_Order_Mod2_Real_1d

Code: `potentials_1d.py:522--616` (post-fix). **Term:** 'Scattering_Fourth_Order_Mod2_Real_Q1' (only the $Q = 1$ wiring exists — there is no plain, fine- Q 'Scattering_Fourth_Order_Mod2_Real' term).

Formula. Identical to Section 5, except the envelope is the **modulus squared** instead of the modulus:

$$E^{(2)}[s](t) := |Wx^{\text{norm}}[s](t)|^2, \quad S_4^{\text{Mod2,Re}}[s_1, s_2, j] = \mathbb{E}_t \left[\text{Re}(W_j E^{(2)}[s_1](t) \cdot \overline{W_j E^{(2)}[s_2](t)}) \right]. \quad (12)$$

j includes the low-pass channel (same default as the non-Mod2 Real class).

Filter bank. `potentials_builder.py:222: Scattering_Fourth_Order_Mod2_Real_1d(J, 1, filters, filters[:, :-1])` — coarse ($Q = 1$) bank for both layers, explicitly. [\[verified from code\]](#)

Normalization. `self.norm[s] =  $\sigma[s]$` [\[fixed 2026-08-31\]](#) — was `self.norm[s] =  $(\mathbb{E}[|Wx[s]|^2])^{-1/4}$` before this session (old line kept commented out). Squaring the envelope changes how many powers of the normalization factor propagate into the final statistic (two, instead of one), but this cancels exactly against the extra power picked up from squaring $|Wx^{\text{norm}}|$ itself — repeating the scale-tracking argument from Section 5 for the squared envelope gives a raw-statistic scaling of $E_{s_1}^{1-2p} E_{s_2}^{1-2p}$, which *also* vanishes only at $p = 1/2$. So the same exponent, 0.5, is correct for both the modulus and modulus-squared families — the previous $-1/4$ value was not merely "right for one, wrong for the other," it did not achieve scale-invariance for *either*.

8 Scattering_Fourth_Order_Mod2_Imag_1d

Code: `potentials_1d.py:618--729`. **Term:** 'Scattering_Fourth_Order_Mod2_Imag_Q1' (again, only the $Q = 1$ wiring exists).

Formula. As Section 7, imaginary part, $s_1 < s_2$ strictly, j excludes the low-pass channel (same `include_lowpass=False` convention as the non-Mod2 Imag class):

$$S_4^{\text{Mod2,Im}}[s_1, s_2, j] = \mathbb{E}_t \left[\text{Im}(W_j E^{(2)}[s_1](t) \cdot \overline{W_j E^{(2)}[s_2](t)}) \right]. \quad (13)$$

Filter bank. `potentials_builder.py:227: Scattering_Fourth_Order_Mod2_Imag_1d(J, 1, filters, filters[:, :-1])` — coarse ($Q = 1$) bank, both layers. [\[verified from code\]](#)

Normalization. `self.norm[s] =  $\sigma[s]$` (exponent 0.5, comment in code: "matches Imag_1d convention (not the $-1/4$ used by the Real classes)"). [\[verified from code\]](#) — already correct before this session; not modified.

9 Scalar

Code: `potentials_1d.py:731--877`. **Terms:** `'Scalar_phi_quantile_confine'`, `'Scalar_morlet_quantile_confine'`, `'Scalar_psi_quantile_confine'`.

Formula. Not a moment — a *soft empirical CDF* of $|Wx[\lambda]|$ at K (`num_centers`, default 20) quantile-spaced locations per channel λ , i.e. it constrains the shape of the marginal distribution rather than a single statistic. Calibrated once in `fit(x)`: centers $c_k[\lambda] = Q_{q_k}(|Wx[\lambda](t)|)$ placed at K quantiles $q_k \in [\text{quantile_min}, \text{quantile_max}]$ (default $[0, 1]$), and widths $s_k[\lambda]$ set from consecutive-quantile spacing times `stride_sigmas` (default 0.75). Then, per center $k < K$:

$$\Sigma[k, \lambda] = \frac{1}{c_k[\lambda]} \mathbb{E}_t \left[\sigma \left(\frac{|Wx[\lambda](t)| - c_k[\lambda]}{s_k[\lambda]} \right) \right], \quad (14)$$

and for $k = K$ (the top quantile, when `confine=True`), an extra polynomial tail correction is added: letting $u(t) = (|Wx[\lambda](t)| - c_K[\lambda])/s_K[\lambda]$, the weight becomes $\sigma(u(t)) \cdot (1 + u(t)^8)$ before the same time-average and $1/c_K[\lambda]$ normalization. This is a genuinely different statistic family from every other class in this file — a histogram/CDF constraint, not a moment.

Filter bank. Three wirings, one per bank:

- Line 142: `Scalar(filters_Phi, ...)` — term `'Scalar_phi_quantile_confine'`: the separate J -many low-pass stack.
- Line 145: `Scalar(filters, ...)` — term `'Scalar_morlet_quantile_confine'`: coarse ($Q = 1$) bank.
- Line 148: `Scalar(filters_Q, ...)` — term `'Scalar_psi_quantile_confine'`: fine bank.

[\[verified from code\]](#)

Normalization. No `fit_micro/self.norm` mechanism at all — normalization here is implicit in the construction itself: the centers and widths $(c_k[\lambda], s_k[\lambda])$ are quantiles of the *same* channel's own data, so the statistic is comparable across scales by construction, and the explicit $1/c_k[\lambda]$ division further rescales each bin by its own center. Not directly comparable to (1)'s convention (there is no single "physical units" this class shares with the moment-based potentials above), but not obviously deficient for its own purpose either — a different question from the one this note otherwise focuses on.

10 L2p_norm

Code: `potentials_1d.py:882--917`. **Terms:** `'L_2'` ($p=1$), `'L_4'` ($p=2$), `'L_6'`, `'L_6_psi'` ($p=3$), `'L_8'` ($p=4$; also `'L_9'` and `'L_10'` — a pre-existing bug overwrites key `'L_8'` instead of adding a new one, so at most one of `L_8/L_9/L_10` ever survives in the returned dict), plus `'L_2_phi'`, `'L_2_lowpass'`, `'L_4_lowpass'`.

Formula. A generic, arbitrary-order *single-scale* even moment of the modulus:

$$L_{2p}[\lambda] = \frac{\mathbb{E}_t[|Wx[\lambda](t)|^{2p}]}{\text{self.norm}[\lambda]}. \quad (15)$$

No cross-scale term of any kind — see the comparison with `Scattering_Third_Order_Real_1d` in the box below.

Filter bank. All plain `L_2...L_10` use **coarse filters** ($Q = 1$); the one exception is `'L_6_psi'`, which uses **fine filters**. Q instead (line 106) — same $p = 3$ formula, finer scale grid. The `_phi` variants use `filters.Phi` (the separate J -many low-pass stack); `L_2_lowpass/L_4_lowpass` slice **filters** down to just its bundled low-pass channel, `filters[:, -1:, :]`. [\[verified from code\]](#)— in your currently-active experiment configs, only `'L_2_lowpass'` is actually used (`gaussian_theta_scaling.ipynb`); none of the others appear in your current **terms** lists.

Normalization. `self.norm[\lambda] = (\mathbb{E}[|Wx[\lambda]|^{2p}])^{-1/2}` (fit from reference data), giving $L_{2p}[\lambda] = \mathbb{E}_t[|Wx[\lambda]|^{2p}] \cdot M_{\text{ref}}[\lambda]^{1/2}$ — on matching data this is $\approx M_{\text{ref}}[\lambda]^{3/2}$, *not* 1. [\[does not match Eq. 1, unresolved\]](#) Two independent problems, not one:

1. Even granting the class's own apparent goal (self-normalize each order to ≈ 1 on its reference data), the active exponent doesn't achieve it — it *inflates* rather than divides down, the same class of mistake found and fixed in Sections 5–7. A commented-out alternative sits right below the active line (`self.norm = raw_moment`, no exponent), which *would* self-normalize correctly.
2. Even that corrected version would not implement (1): it normalizes L_{2p} by *its own* $2p$ -th moment $M_{2p}[\lambda] = \mathbb{E}[|Wx[\lambda]|^{2p}]$, not by $\sigma[\lambda]^{2p} = (\mathbb{E}[|Wx[\lambda]|^2])^p$ as the paper's convention requires. These coincide only at $p = 1$ (that is exactly `L_2`); for $p = 2, 3, 4$ they are genuinely different quantities (e.g. for a Gaussian process, $\mathbb{E}[|Wx|^4] = 2(\mathbb{E}[|Wx|^2])^2$, so `L_4`'s reference differs from σ^4 by more than a relabeling). Self-normalizing each order against its own value also forfeits the paper's actual point: keeping every renormalized order in the *same* units (σ) so their magnitudes stay comparable to each other, not just individually ≈ 1 .

Not fixed in this session — flagged for a separate decision.

11 L2p1_norm

Code: `potentials_1d.py:919--954`. **Terms:** `'L_1'` ($p=0$), `'L_3'` ($p=1$), `'L_5'` ($p=2$), `'L_7'` ($p=3$), plus `'L_1_phi'...``'L_7_phi'`.

Formula. The odd-order counterpart of Section 10:

$$L_{2p+1}[\lambda] = \frac{\mathbb{E}_t[|Wx[\lambda](t)|^{2p+1}]}{\text{self.norm}[\lambda]}. \quad (16)$$

Again purely single-scale; no term in this class can represent a cross-scale (order-3-or-higher scattering) coefficient — see the boxed note below.

Filter bank. All plain L_1...L_7 use **coarse filters** ($Q = 1$); the `_phi` variants use `filters_Phi`. There is no fine-bank ($Q > 1$) wiring for this class anywhere in `get_1d_potentials`. [\[verified from code\]](#)— none of these terms appear in your currently-active experiment configs.

Normalization. `self.norm` $[\lambda] = \mathbb{E}[|Wx[\lambda]|^{2p+1}]$ (the raw reference moment itself, no exponent at all). This *does* self-normalize L_{2p+1} to ≈ 1 on matching data — but that is dividing a statistic by *itself*, which is a different (weaker) claim than the paper’s $/\sigma^{2p+1}$ convention, for the same reason given in point 2 of Section 10: it does not keep different orders in comparable, shared units. [\[does not match Eq. 1, unresolved\]](#)

Summary table

Class	Order	Filter bank(s)	Normalization
Scattering_First_Order_1d	1	fine, band-pass only	none
Scattering_Second_Order_1d	2	fine <i>or</i> coarse	none
Scattering_Second_Order_Bulk_1d	2 (trimmed)	fine	tail-window calib., not (1)
Scattering_Third_Order_Real_1d	3 (cross-scale)	fine, but only correct at $Q=1$	none
Scattering_Fourth_Order_Real_1d	4 (cross-scale)	fine+coarse, or coarse only	$\sigma[s]^{0.5}$ – fixed
Scattering_Fourth_Order_Imag_1d	4 (cross-scale)	fine+coarse, or coarse only	$\sigma[s]^{0.5}$ – already correct
Scattering_Fourth_Order_Mod2_Real_1d	4, mod-squared	coarse only ($Q=1$)	$\sigma[s]^{0.5}$ – fixed
Scattering_Fourth_Order_Mod2_Imag_1d	4, mod-squared	coarse only ($Q=1$)	$\sigma[s]^{0.5}$ – already correct
Scalar	CDF/quantiles	phi-stack, coarse, or fine	implicit in quantile calib.
L2p_norm	even, 2, 4, 6, 8	coarse (mostly), one fine	wrong exponent <i>and</i> wrong reference quantity
L2p1_norm	odd, 1, 3, 5, 7	coarse only	self-normalizes, not σ -based